Ocean Engineering 270 (2023) 113632

Contents lists available at ScienceDirect

Ocean Engineering

journal homepage: www.elsevier.com/locate/oceaneng

A path planning approach for unmanned surface vehicles based on dynamic and fast Q-learning

Bing Hao [a,*], He Du [a], Zheping Yan [b]

[a] College of Computer and Control Engineering, Qiqihar University, Qiqihar, Heilongjiang Province, China
[b] College of Intelligent Systems Science and Engineering, Harbin Engineering University, China

ARTICLE INFO

Handling Editor: Prof. A.I. Incecik

Keywords:
Unmanned surface vehicles
Path planning
Q-learning
Offline
Online

ABSTRACT

Path planning is a critical issue for unmanned surface vehicles (USVs), and an effective path-planning algorithm enables USVs to accomplish the mission. In this paper, a novel algorithm called dynamic and fast Q-learning (DFQL) to solve the path planning problem for USV in partially known maritime environments is proposed, which combines Q-learning with artificial potential field (APF) to initialize the Q-table to provide a priori knowledge from the environment to USV. To accelerate the convergence of Q-learning to the optimal solution and avoid USV's behavior of walking randomly in the early stage of exploration, the static and dynamic rewards are proposed to motivate the USV to move toward the target. Moreover, the performance of the proposed algorithm is verified with offline and online modes for USV in different environmental conditions. By comparing with the existing methods, it shows that the proposed approach is effective for path planning of USV.

1. Introduction

Recently, the increasing attention to marine resources in the world, as well as the rapid development of materials science, mechanical engineering, control theory, sensors, artificial intelligence, and other technologies. Unmanned surface vehicles (USVs), as the name implies remove the human intervention from the platform, and essentially exhibit highly nonlinear dynamics (Liu et al., 2016; Manley, 2008). USVs are widely used in scientific research (Kurowski et al., 2019), ocean resource exploration (Jin et al., 2018), military uses (Sutresman et al., 2017), water rescue (Schofield et al., 2018), environmental missions (Powers et al., 2018) and other applications (Shao et al., 2019; Xue et al., 2020; Yang et al., 2019) for they are reliable, fast, highly maneuverable (Yan et al., 2010). Path planning plays a vital role in the successful completion of USVs' missions due to the complex and volatile nature of the maritime environment. Path planning is the calculation of a feasible path from a start position to a target position in a map or grid without colliding with obstacles on the way. (Patle et al., 2019).

Many current algorithms can implement path planning for USV: A* algorithm (Hart et al., 1968) is one of the most popular search-based path planning algorithms (Dai et al., 2019). The A* algorithm starts from the starting point, checks its adjacent squares, and then extends around until the target is found. The search-based algorithm is easy to apply to path planning task, but the computing time increases exponentially with the size of the map; Rapidly exploring random tree (RRT) (LaValle and Kuffner Jr, 2001) is a sampling-based path planning algorithm. Through collision detection of sampling points in the state space, it avoids spatial modeling and can effectively solve the path planning task of high-dimensional space and complex constraints. However, because the sampling points are randomly generated, the resulting path is not smooth enough and the length is too long. Another type of algorithm is the intelligent algorithm: it is an algorithm that people model by nature-inspired or human mind to imitate solving problems (Goel, 2020), as for instance, particle swarm optimization (PSO) (Eberhart and Kennedy, 1995), genetic algorithm (GA) (Bakdi et al., 2017), and other algorithms. Intelligent optimization algorithm is a kind of algorithm with global optimization performance, strong universality and suitable for parallel processing, but it will fall into local minima and cannot get the optimal path.

All of the above-mentioned algorithms have their advantages and disadvantages, but most of them do not apply to both offline and online path planning (Sung et al., 2021), moreover, the path length and angle are not ideal. When the size of the USV is ignored and treated as a mass point, the searched path is too close to the obstacle, which makes it easy to collide with the obstacle.

To overcome the above limitations, this paper uses an improved Q-

* Corresponding author.
E-mail address: 01522@qqhru.edu.cn (B. Hao).

https://doi.org/10.1016/j.oceaneng.2023.113632
Received 27 February 2022; Received in revised form 15 July 2022; Accepted 4 January 2023
Available online 17 January 2023

learning (Watkins, 1989) based on reinforcement learning to solve the offline and online path planning problem of USV in partially known environments, and calls this algorithm dynamic and fast Q-learning (DFQL). The proposed algorithm fully exploits the advantages of Q-learning in path planning and minimizes its iterations and computation time to enable USVs to avoid obstacles in complex maritime environments at a safe distance both offline and online. The proposed algorithm enables the USV to escape from dead-end paths blocked by obstacles. The effectiveness, superiority, and rapidity of the proposed algorithm are demonstrated through simulations.

1.1. Related work

Path planning, collision avoidance, and energy efficiency for USVs have become a hot topic in recent years, and more and more scholars and researchers are proposing novel ideas and innovative algorithms to solve these problems.

A common approach is to improve on the original algorithm to adapt path planning for USV. Yu et al. (2021) improved the A* and APF algorithm to solve the global and local path planning problem of USVs, to effectively improve system autonomy, increase fault-tolerant resilience, solve low payload capacity and short endurance time. Although the proposed algorithm is validated on simulations and proven to work effectively in different environments, the path length is not optimal and dynamic obstacles in the marine environment are not considered. Yao et al. (2021) developed a novel path planning method based on the D* lite algorithm for real-time path planning of USV in complex environments, this approach is suitable for complex environments with dynamic obstacles. The time of the path planned by this method is effectively reduced compared with the traditional A* and D* algorithms, but the path length and turning angle are not optimal. Another type of approach is to hybridize two or more algorithms, avoiding the limitations of a single algorithm. Liu et al. (2019) hybridized PSO with opposition-based learning (OBL) (Tizhoosh, 2005) to solve the USV global and local path planning problem. The combination of these two algorithms can effectively solve the problems of path optimization and dynamic obstacle avoidance, but the path length and smoothness calculated by this algorithm are not ideal. To effectively improve system autonomy, increase fault-tolerant resilience, solve low payload capacity and short endurance time of USVs, Sang et al. (2021) presented a novel deterministic algorithm named multiple sub-target artificial potential field (MTAPF). The algorithm does not consider dynamic obstacles in the maritime environment.

The aforementioned related papers have achieved good results in terms of reducing the computational effort and path length of the algorithm. However, to the best of our knowledge, none of the previous work specifically targeted the issue of USV simultaneous offline and online modes for the path planning problem. Therefore, this paper proposes a DFQL algorithm to solve this problem. The proposed algorithm can reduce the computation time and number of iterations to converge to the optimal solution compared to the classical Q-learning. Moreover, this algorithm enables the USV to generate feasible safe paths under both offline and online modes with optimal path length and smaller turning angle.

1.2. Original contributions

The DFQL algorithm proposed in this paper is applied to solve the offline and online path planning problems for USVs with the following main contributions.

1. In response to the problem that USVs are disturbed by waves and cannot follow the established path, a safe distance is proposed to avoid collision with obstacles;
2. This paper combines Q-learning with APF to initialize Q-table, to void the defects of random movement around the starting position at the beginning of the algorithm, slow convergence speed, and long computation time; change the constant reward into a combination of static and dynamic reward values to speed up the convergence of the algorithm; improve the algorithm in the complex obstacle environment is easy to fall into a dead-end path blocked by obstacles, unable to reach the target position disadvantage.
3. The proposed DFQL algorithm can plan offline and online path planning to avoid static and dynamic obstacles for USV in maritime environments.
4. By comparing with other algorithms, it is demonstrated that DFQL can complete the path planning task considering path length, collision avoidance, and smoothness.

To better solve the problem and prove the effectiveness of the proposed DFQL algorithm, the remainder of this paper is organized as follows: first, describe the path planning problem in Section 2; then the proposed DFQL algorithm is described in Section 3; the comparative experimental and simulation results for offline and online path planning are presented in Section 4; and finally, conclusions are drawn and future work is discussed in Section 5.

2. Preliminaries

In this section, this paper first describes the USV path planning problem formulated in this paper and then outlines the Q-learning algorithm in preparation for the DFQL algorithm proposed in Section 3.

2.1. Problem formulation

The schematic diagram of USV and the definition of safe distance are shown in Fig. 1 (a), where, x_a *and* y_a *represent the earth-fixed frame,* x_b *and* y_b *represent the body-fixed frame. It is assumed that the origin of the body-fixed frame is located in the USV center of mass. Because of the sea surface working environment due to the existence of wave interference, this paper defines the USV safety distance radius as* r_{sd}*, to avoid collision of the planned path with obstacles. The simulation environment is divided into a 7x7 grid map with side length* $2r_{sd}$*, according to the USV safety distance in a Cartesian coordinate system.* The white grid γ represents the feasible area; the black grid S_{obs} represents the obstacles (land, ship, and other obstacles on the sea surface);

When there are no obstacles in the environment the USV can move in the direction shown in Fig. 1 (b), the USV is defined to be able to move in eight adjacent directions within the environment, and when it is at the edge of the environment the moveable directions all point to the interior of the environment. Fig. 1 (c) shows the marine environment, in which the black part is the obstacle detected by the onboard sensor. Convert the obstacles in Fig. 1 (c) into Fig. 1 (d). When the occupied space does not meet one grid, it is approximately one grid. When there are obstacles around the USV, the moveable directions are shown in Fig. 1 (e); Fig. 1 (f) indicates the starting position $s_{start} = (1,1)$, target position $s_{target} = (7,7)$, this paper aims to plan the safe path of USV from the starting position to the target position.

2.2. Q-learning algorithm

Reinforcement learning is a class of unsupervised algorithms (Caruana and Niculescu-Mizil, 2006). The algorithm emphasizes the learning process of the agent interacting with the environment. The agent's target is to evaluate the action to be chosen in each state of its environment by learning the state-action value function for that state. The agent is not told which action to take, but it must choose an optimal action in each state that maximizes the cumulative reward value (Hao et al., 2022). The USV interacts with the environment through continuous feedback, generating more data (states and returns), and using the new data to further improve its behavior. Q-table is the expectation that $Q(s_t, a_t)$ can gain by taking action a_t in state s_t, updated by the equation

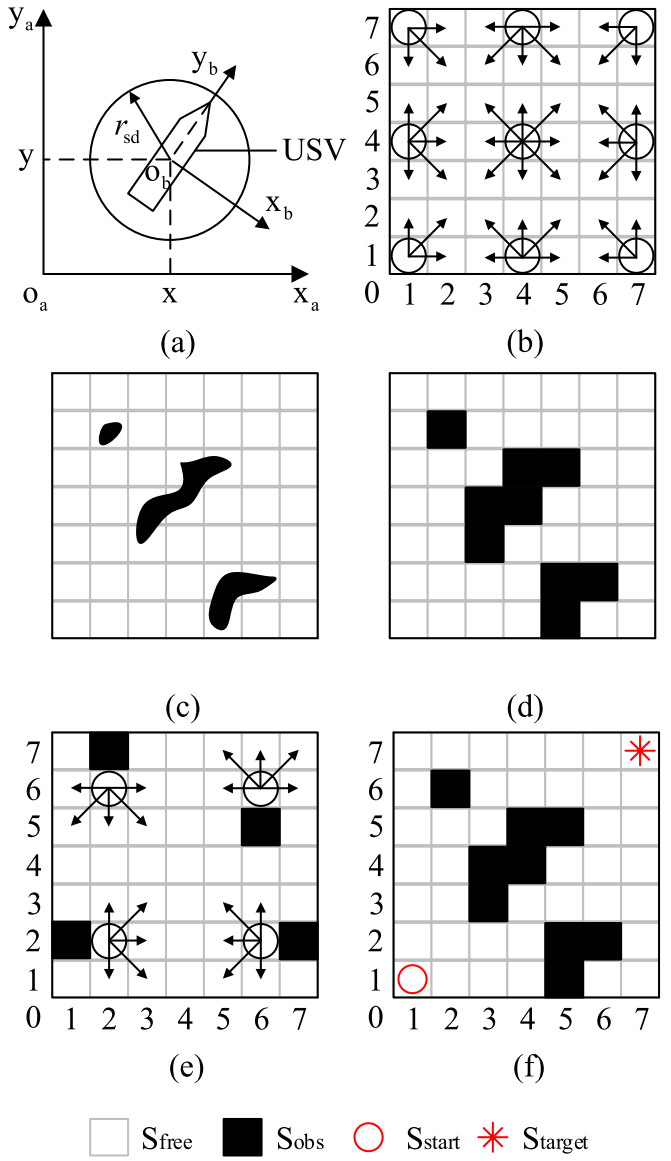

Fig. 1. Path planning problem formulation for USV.

shown in Eq. (1).

$$Q(s_t,a_t)\leftarrow(1-\alpha)*Q(s_t,a_t)+\alpha*\left[r_{t+1}+\gamma\max_a Q(s_{t+1},a_{t+1})\right] \tag{1}$$

The above function can also be written as Eq. (1) and Eq. (2):

$$Q(s_t,a_t)\leftarrow(1-\alpha)*Q(s_t,a_t)+\alpha*\left[r_{t+1}+\gamma V(s_{t+1})\right] \tag{2}$$

$$V(s_t)\leftarrow V(s_t)+\alpha*\left[r_{t+1}+\gamma V(s_{t+1})-V(s_t)\right] \tag{3}$$

$$V^*(s_t)=\max_a Q(s_t,a_t) \tag{4}$$

where, α ($0\le\alpha\le1$) is the learning rate parameter, γ ($0\le\gamma\le1$) is the discount rate parameter.

In this paper, the Q-learning algorithm in reinforcement learning is applied to USV path planning for the following reasons.

- Reinforcement learning has superior interactivity with the environment without the need for positive or negative labels (Tong, 2009). USVs gain current knowledge by exploring and learning from the maritime environment and improving their operational strategies to adapt to the environment.
- It requires few parameters, is easy to implement, and uses a greedy strategy (Chípuli and de la Mota, 2021).
- Q-learning uses off-policy (Haarnoja et al., 2018), the selection of actions according to the target strategy can be applied not only to USV offline planning but also to online path planning.

According to the superior characteristics of Q-learning in path planning and to address its shortcomings, this paper proposes an improved algorithm DFQL based on Q-learning, which combines an artificial potential algorithm with Q-table initialization to provide a priori knowledge for USV and shorten the computation time; a combination of static and dynamic rewards is set to speed up the convergence of the algorithm. The planned path lengths and turn angles are improved compared to the original algorithm, and the computation time is reduced.

3. The proposed DFQL algorithm

In this section, the proposed DFQL algorithm is elaborated in five stages for solving the path planning problem in USV offline and online. To better describe the studied content, the USV path planning problem discussed in this paper has the following premises (Hao et al., 2022).

- The USV and the obstacle environment are three-dimensional objects in practice. This paper ignores the heights of the USV and obstacle, moreover, treats them as objects in two dimensions.
- Ability to accurately position and control the robot to move to a specified position by ignoring the influence of sensor measurement uncertainties or dynamics on the USV during the planning process.
- The USV path planning studied in this paper is divided into two parts: offline and online modes. The location, shape, and size of obstacles in the environment are completely known in advance to the USV in the offline mode; the environment is partially known or completely unknown in the online mode and is equipped with the necessary sensors to identify obstacles during navigation (e.g., touch sensors).

3.1. Q-table initialization

To optimize slow convergence speed and long computation time of Q-learning, this paper uses the artificial potential method combined with the Q-learning algorithm to optimize the initial Q-table. The reasons are: the artificial potential method can be combined with the Q-learning algorithm in both offline and online modes, it is easy to implement in grid maps, provides prior knowledge of the environment for USV, and speeds up the computation and convergence speed.

3.1.1. Q-table initialization for offline path planning

The information in the environment (starting position, target position, obstacle shape, size, and location coordinates) in the offline mode is completely known to USV, so Coulomb's law is used to model the APF for the grid map, as shown in Eq. (5) ~ Eq. (9):

$$U_{a1}(s_t)=\frac{1}{2}k_a\rho_g^2(s_t) \tag{5}$$

$$U_{r1}(s_t)=\begin{cases}\frac{1}{2}k_r\left(\frac{1}{\rho_{ob}(s_t)}-\frac{1}{\rho_o}\right)^2, \rho_{ob}(s_t)\le\rho_o\\ 0 \quad\quad, \rho_{ob}(s_t)\le\rho_o\end{cases} \tag{6}$$

$$U_{n1}(s_t)=U_{r1}(s_t)+U_{a1}(s_t) \tag{7}$$

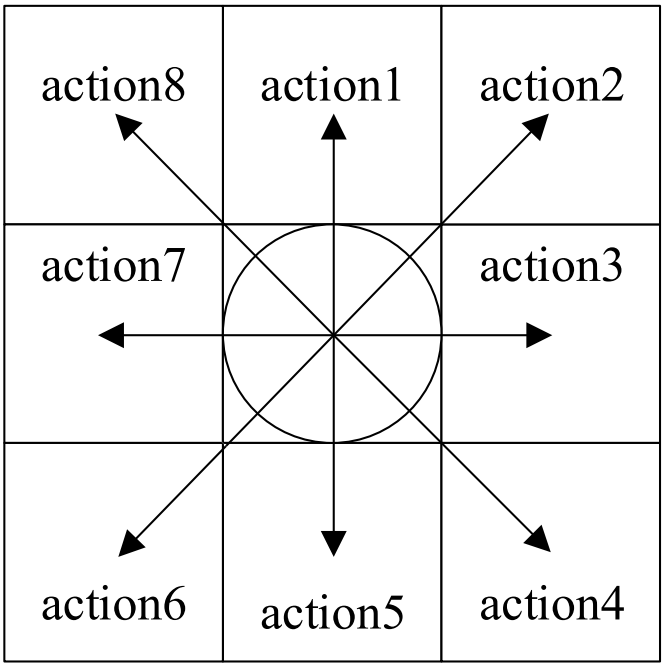

Fig. 2. Actions of USV.

$$U_1(s_t) = \left| \frac{U_{\max\ 1} - U_{n1}(s_t)}{U_{\max\ 1}} \right| \tag{8}$$

$$Q_{01}(s_0, a_0) = U_1(s_t) \tag{9}$$

where, $U_{a1}(s_t)$ is the gravitational field producing gravitational force in state s_t; $U_{r1}(s_t)$ is the gravitational field producing gravitational force in state s_t; $U_{n1}(s_t)$ is the total potential energy of state s_t; $\rho_g(s_t)$ is the Euclidean distance between state s_t and the center of the target position; $\rho_{ob}(s_t)$ is the Euclidean distance between state s_t and the center of the obstacle; ρ_o is the obstacle influence factor; k_a and k_r are the scale factors. $U_1(s_t)$ is the potential energy in state s_t; $U_{\max\ 1}$ is the highest potential energy in state s_t.

3.1.2. Q-table initialization for online path planning

In the online mode, the information in the environment is partially known or completely unknown to the USV, and the USV reaches the target position by detecting the surrounding obstacles through the sensors under the condition that only the starting position and the target position are known, so the repulsive force generated by the obstacles in Section 3.1.1 is ignored and only the gravitational force generated by the target position is calculated to optimize the initial Q-table and model the artificial potential energy, and the equations are shown in Eq. (10) ~ Eq. (12).

$$U_{a2}(s_t) = \frac{1}{2} k_a \rho_g^2(s_t) \tag{10}$$

$$U_{n2}(s_t) = U_{a2}(s_t) \tag{11}$$

$$Q_{02}(s_0, a_0) = U_2(s_t) = \left| \frac{U_{\max\ 2} - U_{n2}(s_t)}{U_{\max\ 2}} \right| \tag{12}$$

where, $U_{a2}(s_t)$ is the gravitational field producing gravitational force in state s_t; $U_{n2}(s_t)$ is the total potential energy of state s_t; $\rho_g(s_t)$ is the Euclidean distance between state s_t and the center of the target position; $\rho_{ob}(s_t)$ is the Euclidean distance between state s_t and the center of the obstacle; k_a is the scale factors. $U_{\max\ 2}$ is the highest potential energy in state s_t.

3.2. Action selection

To avoid obstacles and reach the target position with the shortest path length, eight directions of movement are defined for the USV (Top, Top right, Right, Bottom right, Bottom left, Left, and Top left). According to the safe distance defined in Section 2.1, USV can reach the adjacent 8 grids, as shown in Fig. 2. When the safe distance is defined as r_{sd} = 5m, the corresponding movement and distance are:

action1: a_1 = move up 10m;
action2: a_2 = move up to the right $10\sqrt{2}$m;
action3: a_3 = move right 10m;
action4: a_4 = move down to the right $10\sqrt{2}$m;
action5: a_5 = move down 10m;
action6: a_6 = move down to the left $10\sqrt{2}$m;
action7: a_7 = move left 10m;
action8: a_8 = move up to the left $10\sqrt{2}$m;

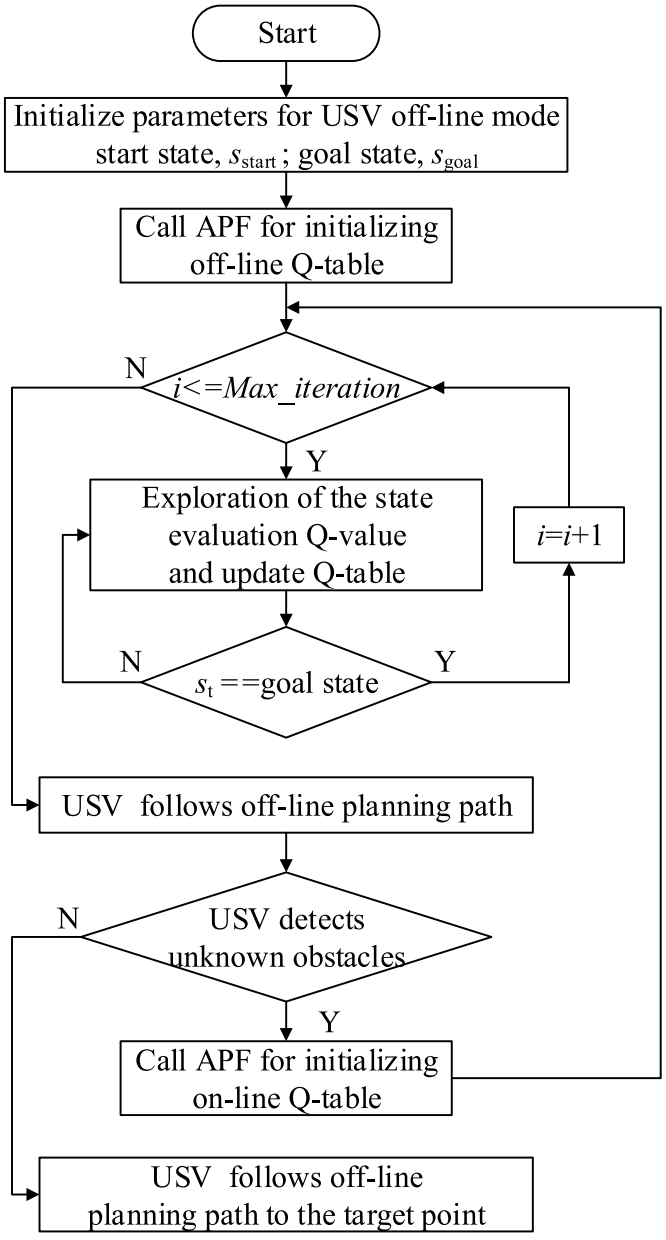

Fig. 3. The flowchart of DFQL.

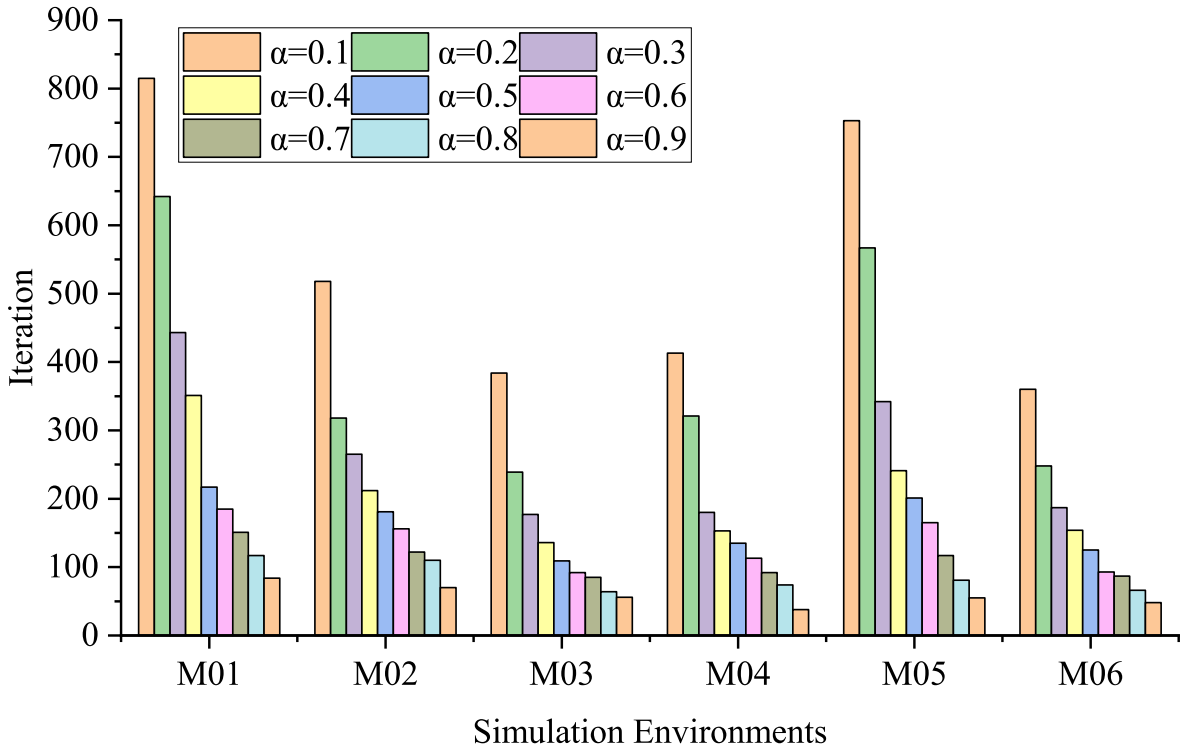

Fig. 4. The number of iterations to converge to the shortest path length in the corresponding environment ($\gamma = 0.9$, $\alpha = 0.1$–0.9).

3.3. Reward function

The reward function is used to determine the value of the behavior, and USV interacts with the environment according to the reward function to adjust the action strategy by the reward value (Jiang et al., 2019). Setting the reward function correctly helps to reinforce the desired behavior and punish the improper behavior. In previous reinforcement learning, the reward value is usually a static constant, which leads to its random search in the environment, resulting in increased convergence time. To reduce the computation, this paper uses Manhattan distance for

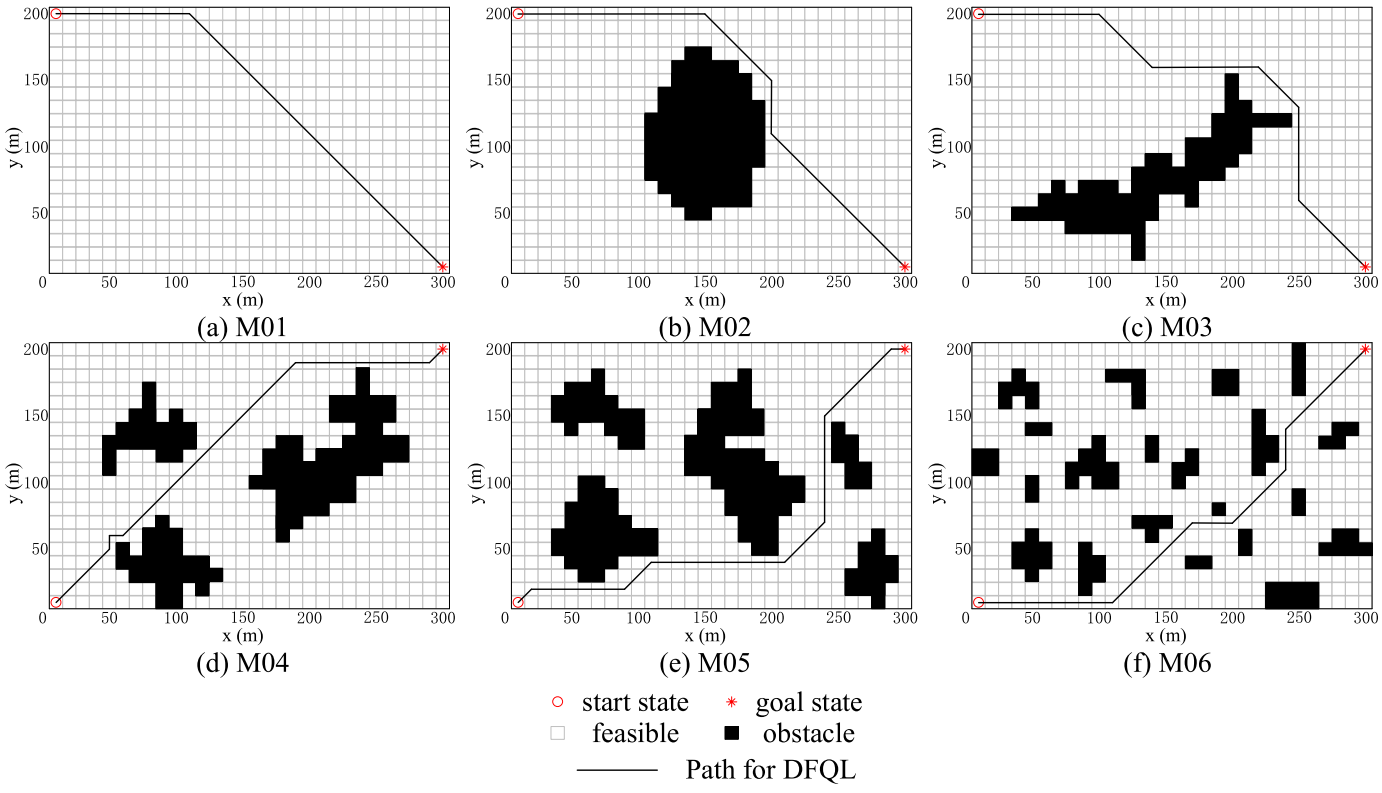

Fig. 5. Path planning (solution) for USV in different test environments.

Table 1
Parameters setting of DFQL, CQL, PSO, GWO, DA, and MFO.

Algorithms	Parameters Selection	*Max*	*Pop*
DFQL	$\alpha = 0.9, \gamma = 0.9, \rho_o = 2, k_a = 1.5, k_r = 1.5$	100	–
CQL	$\alpha = 0.9, \gamma = 0.9$		–
PSO	$c_1 = 2, c_2 = 2$		30
	$\omega = \omega_{max} - Iter * ((\omega_{max} - \omega_{min})/Max)$		
	$\omega_{max} = 0.9, \omega_{min} = 0.2$		
GWO	$a = 2 - Iter * (2/Max), C = 2 * \text{rand}()$		
DA	$a = 2 * \text{rand}() * (0.1 - 0.1 * Iter/(Max/2))$		
	$c = 2 * \text{rand}() * (0.1 - 0.1 * Iter/(Max/2))$		
	$e = 0.1 - 0.1 * Iter(Max/2)$		
	$f = 2 * \text{rand}()$		
	$r = 3 + (Iter/Max)$		
	$s = 2 * \text{rand}() * (0.1 - 0.1 * Iter/(Max/2))$		
	$\omega = \omega_{max} - Iter * ((\omega_{max} - \omega_{min})/Max)$		
	$\omega_{max} = 0.9, \omega_{min} = 0.2$		
MFO	$t = (-1 + Iter * (-1/Max) - 1) * \text{rand}() + 1, b = 1$		

calculation. The reward functions are shown in Eq. (13) ~ Eq. (17).

$$reward = r_s * (1 + r_d) \tag{13}$$

$$r_s = \begin{cases} 1, \text{hor. or ver. movement} \\ 2, s_{t+1} \text{ is the start node} \\ 1/\sqrt{2}, \text{diagonal movement} \\ 10, s_{t+1} \text{is the target node} \\ -\text{inf}, s_{t+1} \text{ is the forbidden node} \end{cases} \tag{14}$$

$$d_t = \left|y_{\text{target}} - y_t\right| + \left|x_{\text{target}} - x_t\right| \tag{15}$$

$$d_{t+1} = \left|y_{\text{target}} - y_{t+1}\right| + \left|x_{\text{target}} - x_{t+1}\right| \tag{16}$$

$$r_d = \frac{d_t - d_{t+1}}{|d_t - d_{t+1}|} \tag{17}$$

where, r_s is the static reward; r_d is the dynamic reward; d_t is the Manhattan distance from the target position in the state s_t; d_{t+1} is the Manhattan distance from the target position in the next state s_{t+1}; (x_t, y_t) is the coordinate in the state s_t; (x_{t+1}, y_{t+1}) is the coordinate in the state s_{t+1}; $(x_{\text{target}}, y_{\text{target}})$ is the coordinate in the target position.

According to this formula to establish the dynamic reward value, when the USV is closer to the target position the reward value is larger, away from the target position the reward value is smaller, avoiding the USV random search process to consume energy and speed up the convergence.

3.4. Convergence for DFQL

Since the update formula of Q-learning has changed, it is necessary to prove the convergence of the proposed algorithm again. Assuming that $Q_n'(s_t, a_t)$ is the estimate of $Q(s_t, a_t)$ for the nth update of the Q-value, define the error of $Q_n'(s_t, a_t)$ as shown in Eq. (18).

$$\delta_n = \left|Q_n'(s_t, a_t) - Q(s_t, a_t)\right| \tag{18}$$

$$\begin{aligned} \delta_{n+1} = \left|Q_{n+1}'(s_t, a_t) - Q(s_t, a_t)\right| \le \left|(1-\alpha)\left(Q_n'(s_t, a_t) - Q(s_t, a_t)\right)\right| + \\ \alpha\gamma \max_a \left|Q_n'(s_{t+1}, a_{t+1}) - Q(s_{t+1}, a_{t+1})\right| \le (1-\alpha)\delta_n + \alpha\gamma\delta_n < \delta_n \end{aligned} \tag{19}$$

As the DFQL algorithm iterates, the Q-value converges to a definite value. USV will move from the starting point to the goal point with the optimal action strategy.

3.5. DFQL for offline and online path planning

In this paper, the proposed DFQL algorithm is applied to the path planning problem of USV offline and online, and its basic ideas are as follows: based on the interaction between the Q-learning algorithm and the environment, the APF is used to initialize the Q table in offline and online modes respectively, to provide the prior knowledge in the environment to the USV, to avoid random selection actions at the early stage of exploration, to improve the efficiency of environment exploration, and to speed up convergence and computation; the static and dynamic reward values are combined to optimize the reward function, to induce the USV to move toward the target position, to improve the computational efficiency of the original algorithm, and to output the shortest path.

To sum up, the flow of USV using DFQL algorithm offline and online modes is as follows: when USV is located at the starting position, offline path planning is performed for known obstacles in the environment; USV follow the offline path when the shipboard sensors detect unknown obstacles near USV, it switches to online mode to avoid the unknown obstacles. After that, it switches to offline mode, if it detects that there are unknown obstacles around the USV, repeat the above steps until the USV reaches the target position, the flow chart is shown in Fig. 3.

4. Simulation results

In Section 4.1, the evaluation function for testing the goodness of USV path planning results is introduced; in Section 4.2, the optimal α andγ values of DFQL are determined based on the path length by testing and analyzing on six different grid maps; in Section 4.3, to demonstrate the effectiveness and generalizability of the proposed algorithm, the simulation experiments of DFQL and the comparison algorithm are conducted on different grid maps under offline and online modes, respectively. The path length, turning angle, and computation time are recorded; in Section 4.4, this paper proves that the proposed DFQL algorithm can be applied to the path planning problem in both offline and online modes of USV by analyzing the experimental results.

4.1. Evaluation function

4.1.1. Path length

This paper defines the path length of the mobile USV from the starting position to the target position as shown in Eq. (20).

$$Path\ Length\ (\text{m}) = \sum_{i=0}^{n} \sqrt{(y_{i+1} - y_i)^2 + (x_{i+1} - x_i)^2} \tag{20}$$

where, $i = 0, 1, 2, \ldots, n$, when $i = 0$, the USV is at the starting position $S = (x_o, y_o)$, when $i = \text{n}$, the USV is at the target position $T = (x_n, y_n)$, (x_i, y_i) represents the coordinates of the current state of the USV; (x_{i+1}, y_{i+1}) represents the coordinates of the next state of the USV.

4.1.2. Angle

The turning angle is the sum of the change in heading angle of USV from the start to the target. This paper defines the turning angle of USV from the starting position to the target position as shown in Eq. (20) ~ Eq. (23).

$$a_i = \sqrt{(y_i - y_{i-1})^2 + (x_i - x_{i-1})^2} \tag{21}$$

$$b_i = \sqrt{(y_{i+1} - y_i)^2 + (x_{i+1} - x_i)^2} \tag{22}$$

$$c_i = \sqrt{(y_{i+1} - y_{i-1})^2 + (x_{i+1} - x_{i-1})^2} \tag{23}$$

$$Angle\ (\text{rad}) = \sum_{i=1}^{n} \left(\pi - ar\cos\frac{a_i^2 + b_i^2 - c_i^2}{2 * a_i * b_i}\right) \tag{24}$$

where, $i = 0, 1, 2, \ldots, n$, (x_i, y_i) represents the coordinates of the current state of USV; (x_{i+1}, y_{i+1}) represents the coordinates of the next state of USV.

4.1.3. CPU time

CPU time is consumed by the algorithm through iterative computation is denoted as *Time* (s). The shorter computation time, the smaller waiting time for USV to execute the action.

4.2. Configuration of environment

In this paper, the maritime environment in which USV works is simplified using a rectangle with 30 grids long and 20 grids wide, denoted as M. The simulation map in this paper is represented by M01 to M06. A circle of radius $r_{sd} = 5$m is used to represent the safe distance of the USV, and a square of the side $2r_{sd} = 10$m is used to represent each grid in the workspace, with the white grid indicating the feasible area and the black grid indicating the obstacle. The center of each grid is marked by a Cartesian coordinate system, with the x-axis indicating the horizontal direction and the y-axis indicating the vertical direction. The coordinate is expressed as (x,y). The first and second dimensions of the raster map represent the x-horizontal and y-vertical coordinates of the grid in the map, respectively. USV understands the environment by assigning values to the elements in the map M, where the obstacles are assigned to 1 and the free space is assigned to 0.

4.3. Parameters selection

Before the DFQL algorithm is executed, the values of the two parameters: learning rate α ($0 \leq \alpha \leq 1$) and decay rate γ ($0 \leq \gamma \leq 1$) in the Q-value update equation Eq. (3) need to be determined based on the evaluation function of Section 4. According to Watkins et al. (Watkins and Dayan, 1992; Watkins, 1989), when the α value is small, the agent goes through all states in the environment and calculates all possible

Table 2
A comparison between DFQL, CQL, PSO, GWO, DA and MFO.

Env	Statistics		DFQL	CQL	PSO	GWO	DA	MFO
M01	Path Length	Mean	369.53	376.51	440.75	371.82	467.11	451.30
		Std. Dev.	3.34	17.83	16.00	7.80	5.84	9.89
		t-test	–	4.34e-2	1.10e-22	1.44e-1	6.61e-51	3.46e-32
	Angle	Mean	0.33	0.75	0.81	1.03	0.39	0.54
		Std. Dev.	0.30	0.60	0.45	0.50	0.30	0.26
		t-test	–	1.47e-3	1.46e-5	4.60e-8	4.50e-1	5.58e-3
	Time	Mean	13.29	13.95	2.93	3.10	3.67	2.71
		Std. Dev.	0.70	0.38	0.35	0.28	0.34	0.24
		t-test	–	4.87e-5	1.99e-46	9.31e-43	2.44e-44	2.03e-41
M02	Path Length	Mean	392.57	402.22	401.50	392.72	393.69	423.76
		Std. Dev.	1.67	9.77	20.84	1.79	3.05	26.49
		t-test	–	8.64e-6	2.63e-2	7.42e-1	8.33e-1	4.67e-7
	Angle	Mean	1.07	1.30	1.71	1.49	1.83	1.11
		Std. Dev.	0.36	0.42	0.49	0.21	0.25	0.49
		t-test	–	2.49e-2	3.62e-7	1.16e-6	4.32e-13	7.10e-1
	Time	Mean	14.23	15.16	3.54	3.52	5.39	3.34
		Std. Dev.	1.32	1.15	0.39	0.15	0.44	0.73
		t-test	–	5.11e-3	3.56e-31	9.32e-29	4.78e-29	7.12e-37
M03	Path Length	Mean	410.68	423.96	432.75	413.22	418.49	441.34
		Std. Dev.	3.11	22.01	25.09	10.50	14.49	21.24
		t-test	–	2.68e-3	4.36e-5	2.13e-1	6.97e-3	9.40e-9
	Angle	Mean	1.23	1.48	1.18	2.01	1.85	0.84
		Std. Dev.	0.45	0.33	0.69	0.42	0.54	0.57
		t-test	–	1.78e-2	7.4e-1	3.35e-9	9.68e-6	5.57e-3
	Time	Mean	13.81	14.62	4.06	3.57	3.87	3.29
		Std. Dev.	1.38	1.50	0.35	0.44	0.40	0.32
		t-test	–	3.27e-2	1.91e-28	2.82e-30	2.22e-29	3.00e-29
M04	Path Length	Mean	375.34	383.35	462.23	377.10	471.41	461.84
		Std. Dev.	2.54	16.04	3.26	3.67	6.29	4.95
		t-test	–	1.12e-2	4.80e-67	3.57e-2	1.18e-43	7.25e-50
	Angle	Mean	1.53	1.96	0.58	1.83	0.57	0.55
		Std. Dev.	0.09	0.52	0.19	0.37	0.17	0.19
		t-test	–	1.09e-4	3.01e-26	1.74e-4	1.06e-28	9.91e-27
	Time	Mean	13.76	14.76	3.39	3.35	5.38	2.62
		Std. Dev.	1.37	1.50	0.40	0.29	0.60	0.10
		t-test	–	8.95e-3	4.44e-30	7.18e-29	2.68e-29	1.81e-28
M05	Path Length	Mean	416.54	423.76	432.36	451.30	438.60	445.83
		Std. Dev.	2.22	11.37	19.56	23.50	22.71	20.36
		t-test	–	1.78e-3	1.28e-4	5.97e-9	1.05e-5	1.04e-8
	Angle	Mean	1.78	2.02	1.40	0.98	1.36	0.97
		Std. Dev.	0.08	0.29	0.56	0.58	0.60	0.58
		t-test	–	1.17e-4	1.09e-3	2.18e-8	7.18e-4	2.12e-8
	Time	Mean	14.09	15.13	3.21	3.21	4.73	3.84
		Std. Dev.	1.83	0.86	0.14	0.16	0.58	0.36
		t-test	–	7.41e-3	1.54e-24	1.37e-24	9.18e-25	1.27e-24
M06	Path Length	Mean	392.91	399.16	414.00	419.66	419.66	403.85
		Std. Dev.	10.29	18.23	11.61	12.98	12.98	10.77
		t-test	–	1.09–1	5.63e-10	3.63e-12	3.63e-12	1.70e-4
	Angle	Mean	1.53	1.73	2.02	2.12	2.14	2.08
		Std. Dev.	0.36	0.54	0.28	0.33	0.45	0.30
		t-test	–	1.14e-1	3.78e-7	1.85e-8	3.68e-7	3.90e-8
	Time	Mean	14.25	14.67	5.64	6.83	10.48	5.84
		Std. Dev.	1.10	0.88	1.06	1.82	1.98	0.96
		t-test	–	1.03e-1	1.13e-37	5.00e-24	8.30e-12	8.26e-38

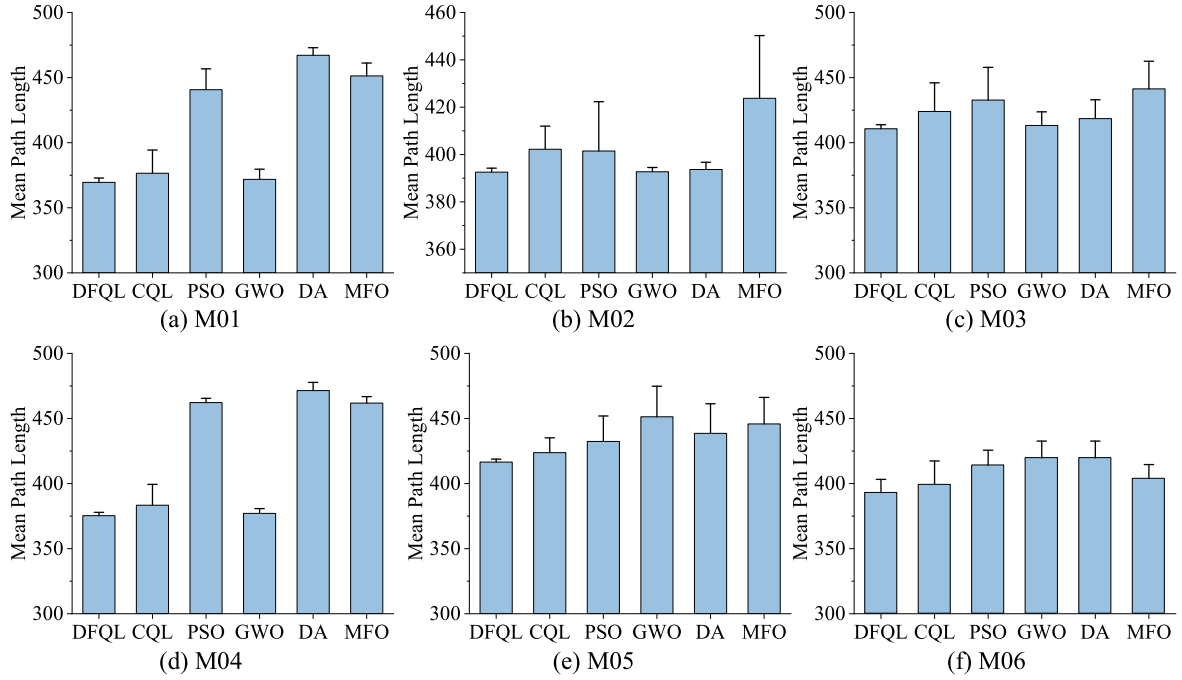

Fig. 6. Compared mean path length of various path planning algorithms for USV in offline mode. (The interval on each bar denotes the standard deviation of the path length).

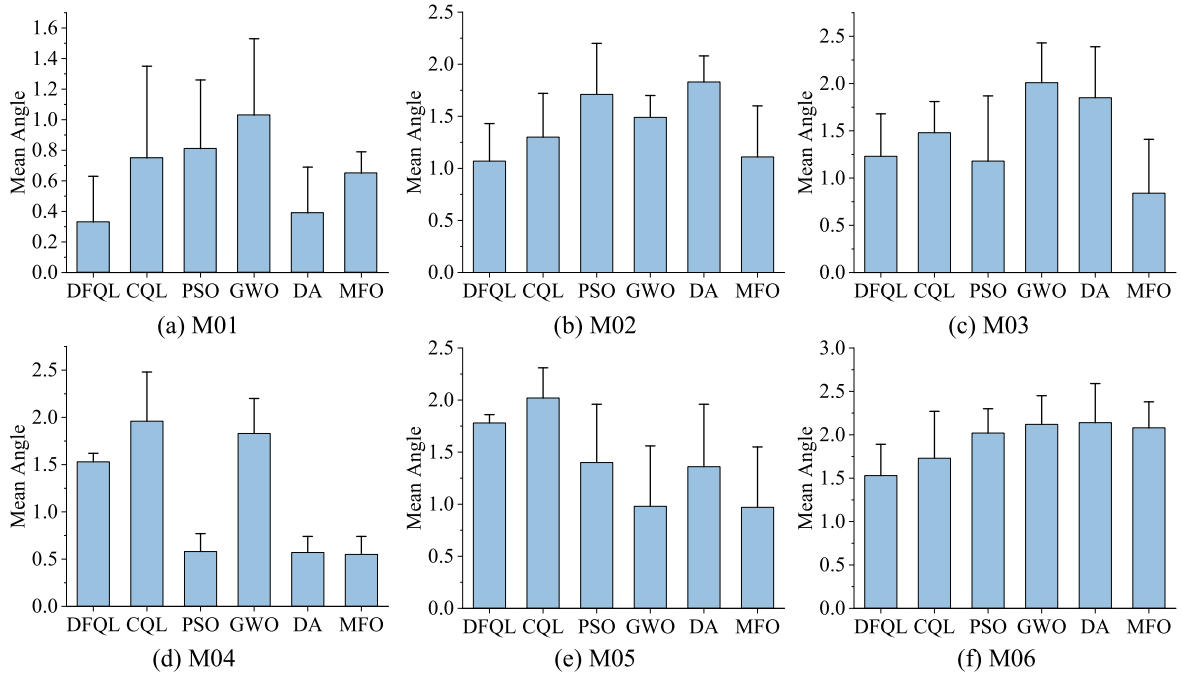

Fig. 7. Compared mean angle of various path planning algorithms for USV in offline mode. (The interval on each bar denotes the standard deviation of the angle).

Table 3
Comparison of algorithm performance.

	Path length vs CQL	Angle vs CQL	Time vs CQL
M01	1.85%	55.56%	4.70%
M02	2.40%	17.95%	6.13%
M03	3.13%	16.95%	5.57%
M04	2.09%	21.70%	6.80%
M05	1.70%	11.98%	6.88%
M06	1.57%	11.11%	2.91%

Table 4
Comparison of algorithm precocity.

	DFQL	CQL	PSO	GWO	DA	MFO
M01	0	0	3	0	9	0
M02	0	0	0	0	0	0
M03	0	0	1	0	0	0
M04	0	0	1	0	16	1
M05	0	0	2	15	7	2
M06	0	0	0	0	0	0

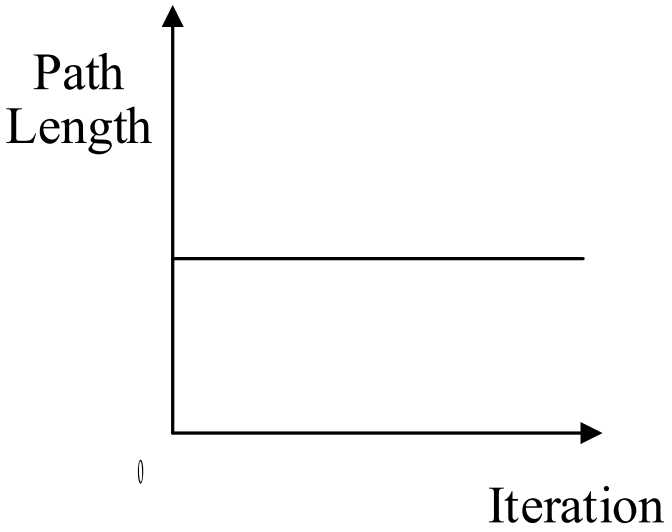

Fig. 8. Precocity of algorithm.

actions, and the Q value converges to the optimal value. When the γ value is large, it can expand the exploration range of the agent and prevent the agent from falling into the problem of local optimum. Therefore, based on the above theory, when $\gamma = 0.9$, this paper records the number of iterations that path length converges to the optimum. Repeat the test 30 times to take the average value, the experimental results are shown in Fig. 4. It can be seen from Fig. 4 that the value of α is inversely proportional to the number of iterations for the path to converge to the optimal value and the computation time for convergence to the optimal value, so considering the computation time of USV, both α and γ are taken as 0.9.

4.4. Offline and online path planning for USV

In this paper, the DFQL algorithm is proposed to solve the USV offline and online path planning problems and the two are combined to work together so that the USV can better plan the path from the start position to the target position with short planned paths and high route smoothness.

4.4.1. Offline path planning for USV

The obstacles within the environment in offline path planning are completely known to the USV, and the objective is to compute a feasible path between the starting start state and the target state. To evaluate the DFQL algorithm proposed in this paper, the simulation environment and the experimental results M01 to M06 are designed as shown in Fig. 5, simulating the USV in different working environments with locations, such as archipelagoes, rivers, and harbors. From simple to complex environments, numerous challenges in the field of path planning are covered: problems such as planning shortest paths over long distances, trap points due to local minima. To fully test the performance of the DFQL algorithm, the algorithm was evaluated based on the evaluation function introduced in Section 4.1. Since the results obtained from each experiment were not identical, each algorithm was repeated 30 times on six ocean simulation environments, recording the path length, turn

Table 5
Parameters setting of DFQL, CQL, RRT and APF.

Algorithms	Parameters Selection	*Max*
DFQL	$\alpha = 0.9, \gamma = 0.9, \rho_o = 2, k_a = 1.5, k_r = 1.5$	100
CQL	$\alpha = 0.9, \gamma = 0.9$	100
RRT	$p = 0.5, size = 20, attempt = 10000$	–
APF	$\rho_o = 2.5, k_a = 2, k_r = 2$	–

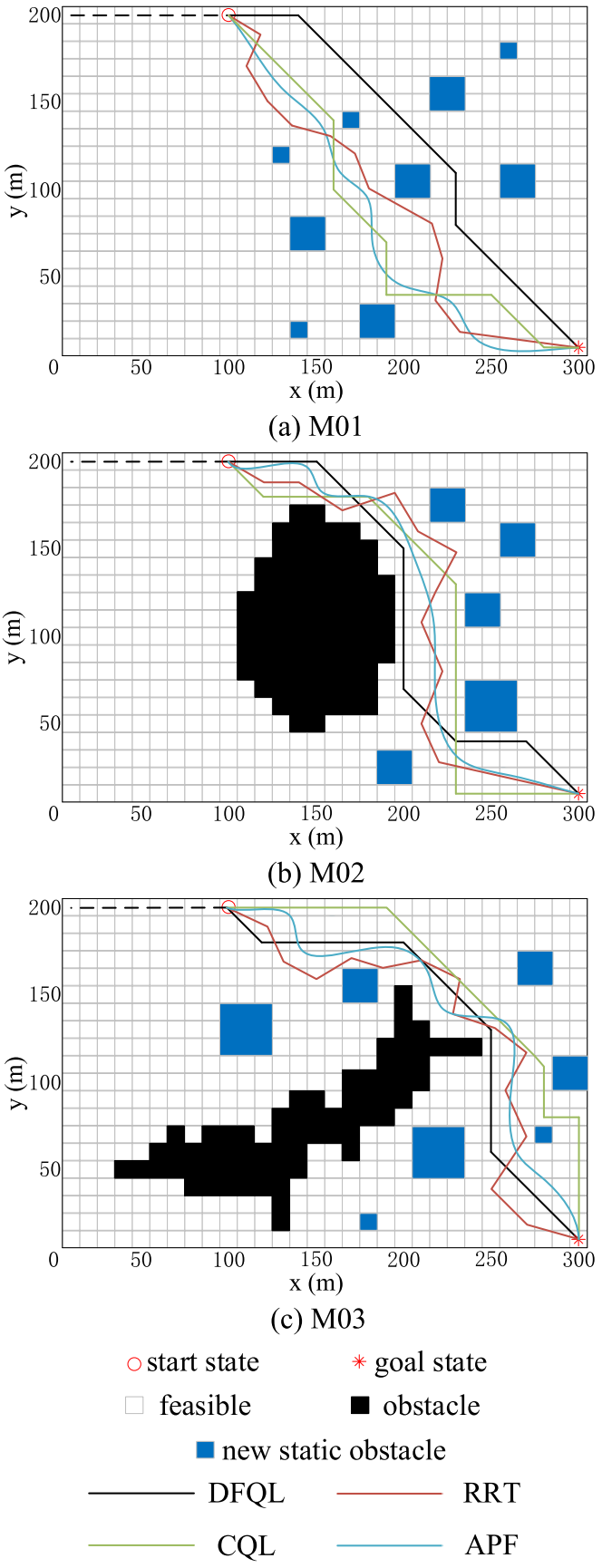

Fig. 9. Path planning (solution) for USV in online mode with static obstacles.

angle, and computation time each time, the mean, standard deviation, and *t*-test (whether there is a significant difference between DFQL and other algorithms) are shown in Table 1.

To evaluate the generality and universality of the DFQL algorithm applied to the USV path planning problem, the DFQL algorithm is compared with CQL (Classical Q-learning) (Watkins, 1989), PSO (Particle Swarm Optimization) (Eberhart and Kennedy, 1995), GWO (Grey Wolf Optimization) (Mirjalili et al., 2014), DA (Mirjalili and Applications, 2016) and MFO (Moth-Flame Optimization) (Mirjalili, 2015)in different simulation environments, respectively. PSO is the basic path planning comparison algorithm. It originates from the study of bird flock predation behavior, which is characterized by easy implementation; GWO is inspired by the leadership hierarchy and hunting mechanism of the grey wolf in nature, which has the characteristics of strong convergence performance and few parameters; DA and MFO are the state-of-the-art algorithms. DA is inspired by the static and dynamic swarming behaviors of dragonflies in nature, which has the advantages of strong stability, good search speed, and global searchability, etc. MFO simulates the special navigation mechanism of moths using lateral positioning during night flight, which has the performance characteristics of strong parallel optimization ability, global superiority, and not easy to fall into local extremes.

To test the DFQL algorithm proposed in this paper under the same conditions, the parameter settings of the compared algorithms are shown in Table 2.

Where, *Max* is the maximum number of iterations, *Pop* is the population size; PSO: c_1 and c_2 are the learning factor, ω is the linearly decreasing weight (LDW) (Tian et al., 2018), $\omega_{\max}$ is the maximum inertia weight, $\omega_{\min}$ is the minimum inertia weight; GWO: a is a constant, the initial value is 2, and decreases linearly from 2 to 0 with the iteration of the algorithm, C is a random number between 0 and 2; DA: a is the alignment weight, c is the cohesion weight, e is the natural enemy weight factor, f is the prey weight factor, r is the neighborhood radius, and s is the separation weight; MFO: t is the path coefficient, b is the logarithmic spiral shape constant; rand() denotes a random number between [0,1], and *Iter* denotes the current iteration number.

The optimal (suboptimal or optimal under optimal conditions) results of DFQL in 30 repeated tests (the path with the shortest turn angle and computation time under the same path length condition is taken) are shown in Fig. 5 (the paths of other compared algorithms are not indicated in Fig. 5 for a clearer representation of the paths of DFQL). From Fig. 5, it can be seen that in the six test maps, the DFQL can effectively solve the path planning problem of USV from the starting position to the target position without collision with obstacles under offline conditions, and it can achieve excellent results in each environment.

Combining the data in Table 2 with Figs. 6 and 7, it can be seen that: the proposed algorithm outperforms the CQL, PSO, GWO, DA and MFO algorithms in terms of path length and turn angle, but the computation time is longer than PSO, GWO, DA and MFO and less than the CQL algorithm. The percentage improvement of DFQL compared to CQL is shown in Table 3, and check whether there is a significant difference between DFQL algorithm and other comparison algorithms by *t*-test (considering 0.05 significance level), where the *t*-test data greater than 0.05 are marked in red

The number of algorithms precocity in 30 iterations of the experiment is recorded in Table 4, and the results show that the DFQL algorithm does not have precocity and can calculate the optimal solution, and it can be further seen that PSO, GWO, DA, and MFO are more prone to precocity in the environment with sparse obstacles, and there is no precocity in the environment with dense obstacles such as Fig. 5(f), and the precocity is shown in Fig. 8. It shows that the path length does not decrease with the increase of iterations.

In summary, the DFQL algorithm proposed in this paper can effectively solve the USV offline path planning problem, and the calculated path length and turning angle are optimal results, which can effectively plan a path in complex environments, and has a large adjustment from path length, turning angle and computation time compared with the original algorithm, but has the disadvantage of longer computation time compared with other algorithms. Test results under nine different cases in Fig. 5 as shown in Table 2 validate that.

Table 6
A comparison between DFQL, CQL, RRT and APF for USV in online mode with several static obstacles.

Env	Statistics		DFQL	CQL	RRT	APF
M01	Path Length	Mean	297.86	309.20	352.94	321.20
		Std. Dev.	4.57	9.91	11.51	
		t-test	–	1.21e-6	9.22e-25	
	Angle	Mean	0.96	1.35	2.02	1.73
		Std. Dev.	0.21	0.49	0.16	
		t-test	–	2.24e-4	8.61e-29	
	Time	Mean	12.42	12.87	6.57	1.86
		Std. Dev.	0.79	0.86	0.94	0.42
		t-test	–	3.96e-2	4.61e-33	2.35e-35
M02	Path Length	Mean	331.20	341.57	382.66	371.75
		Std. Dev.	8.79	10.97	20.17	
		t-test	–	1.66e-4	1.12e-15	
	Angle	Mean	1.37	1.53	2.25	1.92
		Std. Dev.	0.22	0.30	0.26	
		t-test	–	2.35e-2	4.44e-20	
	Time	Mean	13.62	13.98	5.36	2.80
		Std. Dev.	0.85	0.88	0.93	0.29
		t-test	–	1.14e-1	3.71e-41	6.86e-39
M03	Path Length	Mean	323.58	334.13	377.65	379.29
		Std. Dev.	6.28	10.47	13.54	
		t-test	–	2.02e-5	1.29e-22	
	Angle	Mean	1.18	1.29	3.08	3.88
		Std. Dev.	0.24	0.34	0.56	
		t-test	–	1.26e-1	1.99e-24	
	Time	Mean	12.87	13.55	5.83	3.03
		Std. Dev.	0.98	0.54	1.53	0.34
		t-test	–	1.88e-3	1.61e-26	2.35e-35

Table 7
A comparison between DFQL, CQL, RRT and APF for USV in online mode with several nun-static obstacles.

Env	Statistics		DFQL	CQL	RRT	APF
M04	Path Length	Mean	200.64	201.74	228.27	219.39
		Std. Dev.	3.37	5.47	12.40	
		t-test	–	3.51e-1	2.03e-13	
	Angle	Mean	0.63	0.70	1.87	1.57
		Std. Dev.	0.19	0.21	0.27	
		t-test	–	2.09e-1	2.74e-26	
	Time	Mean	10.87	11.88	3.07	1.43
		Std. Dev.	0.94	1.14	0.53	0.30
		t-test	–	4.37e-4	6.31e-37	8.59e-35
M05	Path Length	Mean	314.79	320.30	354.00	336.66
		Std. Dev.	4.60	10.37	14.68	
		t-test	–	1.12e-2	8.69e-16	
	Angle	Mean	1.43	1.54	2.98	1.72
		Std. Dev.	0.22	0.24	0.62	
		t-test	–	5.28e-2	4.79e-15	
	Time	Mean	13.23	14.21	3.16	2.74
		Std. Dev.	0.75	1.13	0.43	0.41
		t-test	–	2.47e-4	6.02e-47	9.08e-47
M06	Path Length	Mean	274.31	279.55	316.42	307.54
		Std. Dev.	4.31	8.52	19.04	
		t-test	–	4.38e-3	3.33e-13	
	Angle	Mean	1.61	1.73	2.93	1.83
		Std. Dev.	0.14	0.21	0.49	
		t-test	–	1.53e-1	7.97e-16	
	Time	Mean	13.37	14.20	4.04	2.97
		Std. Dev.	0.78	0.95	1.29	0.62
		t-test	–	4.84e-4	4.25e-35	1.33e-50

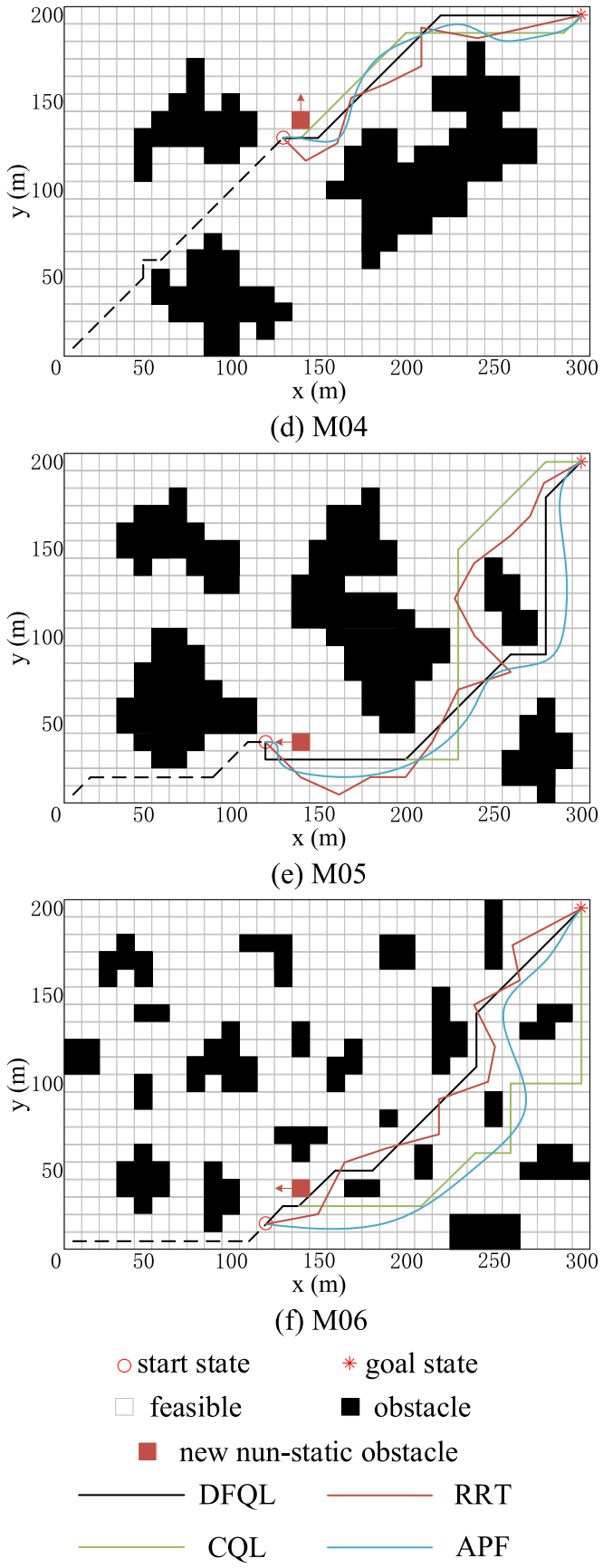

Fig. 10. Path planning (solution) for USV in online mode with nun-static obstacles.

1) Our approach has the shortest path length and smallest turning angle of USV for global path planning in all cases, compared with the comparison algorithms in this paper.
2) The proposed approach has the smallest standard deviations among the comparison algorithms in all cases, which demonstrates the superiority and stability of the DFQL algorithm under different environments.
3) From the time results in Table 2, it can be seen that DSQL has improved compared to CQL, but the computing time slightly higher than PSO, GWO, DA and MFO.

(Each map shows the best path obtained by the DFQL algorithm in offline mode).

4.5. Online path planning for USV

The previous USV path planning problem is based on the environment is completely known, however, in practice the environment is not completely known, there are partially known or completely unknown. In dynamic and uncertain ocean environments, it is difficult or even impossible to obtain the information of various obstacles before path planning (Cheng et al., 2021). This requires USV to be equipped with sensors (radar, lidar, electro-optical (EO) camera, infrared (IR) camera systems, etc.) to obtain information on the position, heading and speed of the surrounding vessels (Han et al., 2020). To test that the proposed DFQL algorithm can solve the USV in path planning problem, this paper sets up static and dynamic obstacles in partial known environments for online planning based on the previous Section 4.3.

To demonstrate the effectiveness and generalizability of the proposed DFQL algorithm in online mode, the DFQL algorithm is analyzed and compared with the commonly used USV online path planning algorithms: RRT and APF. The parameters are set as shown in Table 5. For RRT, p is the random adoption probability; *size* is the step length; *Max_attempt* is the maximum number of sampling. For APF, ρ_o is the obstacle influence factor; k_a and k_r are the scale factors.

4.5.1. Online path planning with static obstacles

This section is tested from the fully known Fig. 5 marine environment in Section 4.3. The actual working process of USV plans the path through the partially known environment, and during the driving process detects the obstacles by sensors and turns to online planning for real-time avoidance of obstacles, which are considered in this section as randomly placed stationary obstacles on the offline path (in the actual marine environment can be stationary ships, floating objects, reefs, and other obstacles).

To describe the experimental procedure, the following steps were used to design the simulation experiment.

1. The USV plans the path offline according to the completely known environment in Section 4.3. The simulation result shows that the path length and turn angle planned by the algorithm proposed in this paper are the shortest, so the USV drives according to the path planned by DFQL.
2. Adopt (a)~(c) in Fig. 5 for online path planning. When USV arrives at coordinates (100,200) in Fig. 9, sensors detect the change of obstacle in the environment and USV switches to online mode, the dotted line in Fig. 9 represents the path of USV traveling according to DFQL in offline mode.
3. In Fig. 9 (a)~(c) the feasible paths are planned with (100,200) as the starting position and (300,1) as the target position for real-time detection, and in order to verify the effectiveness of the proposed algorithm, comparison experiments are conducted with CQL, RRT and APF.

The simulation experiments are repeated 30 times in Fig. 9 (a)~(c), respectively, and the obtained path results are shown in Fig. 9 (a)~(c),

the black grid is the obstacle with completely known environment in offline mode, and the blue grid is the new obstacle detected by the USV sensor, and the experimental data are shown in Table 6. Among them, the RRT results are different for each experiment, and the path length and turning angle are the same in each experimental result of APF, so there is no mean, standard deviation and *t*-test for path length and turning angle (see Table 7).

Repeating the experiment 30 times shows that the DFQL proposed in this paper is significantly better than the comparison algorithm in online path planning for USV (path length and computation time), but the computation time is longer. In the experimental process, the randomness of the path length and turning angle derived by the RRT algorithm is too large (standard deviation) and is not suitable for USV path planning. For APF, although the stability of the path length and turning angle derived each time is good, requires adjusting the corresponding parameters to avoid the problem of falling into local minima. APF is not able to calculate the results when the obstacles are too dense. The paths calculated by RRT and APF calculates the path too close to the obstacle is not conducive to USV planning path.

4.5.2. Online path planning with non-static obstacles

The obstacles in the real maritime environment are not completely stationary, there are dynamic obstacles under the action of moving ships and sea current climate. This case sets up a path planner based on DFQL algorithm according to the dynamic obstacles in online mode. To describe the experimental process (similar to section 4.4.1), the following steps are used to design the simulation experiment.

1. The USV plans the offline path according to the completely known environment in Section 4.3. Simulation results in Section 4.3 show that the path length and turning angle planned by the algorithm proposed in this paper are the shortest, so the USV drives according to the path planned by DFQL.
2. Adopt (d)∼(f) in Fig. 5 for online path planning, when USV travels to (130,130), (120,40) and (120,20) in (d)∼(f) in Fig. 10 respectively, dynamic obstacles are detected through sensors, and the obstacles in the environment are found to change and switch to online mode to avoid dynamic obstacles, the dashed line in Fig. 10 represents the path of driving according to DFQL in the offline mode.

This paper simplifies the motion of dynamic obstacles as uniform linear motion (one grid movement at a time), and set up dynamic obstacles in (d)–(f) in Fig. 10 to verify the effectiveness of the proposed algorithm, and conduct comparison experiments with CQL, RRT and APF. The simulation experiments are repeated 30 times in Fig. 10(d)∼(f), respectively, and the obtained path results are shown in Fig. 10(d)∼(f), the black grid is the obstacle with completely known environment in offline mode, the red grid is the dynamic obstacle detected by USV sensor, the red arrow represents the movement direction of the obstacle. The experimental data are shown in Table 6. Each experimental results are different for RRT, Each experimental results in the path length and turning angle are the same for APF, so the path length and turning angle without mean, standard deviation and *t*-test。

Dynamic obstacles move in different directions in Fig. 10 (d)∼(f) and the experiment is repeated 30 times to show that the DFQL algorithm proposed in this paper can effectively avoid dynamic obstacles when applied to USV online path planning, and the path length and computation time are significantly better than those of the comparison algorithm, but there is a problem of long computation time. In the experimental process, the randomness of the path length and turning angle derived by the RRT is too large (standard deviation), and the algorithm is not able to calculate effective results when the obstacles are set densely, or there is a problem of local minima. The APF algorithm, although the stability of the path length and turning angle derived each time is good, the corresponding parameters need to be adjusted to avoid falling into the problem of local minima, and the algorithm is not able to calculate effective results. When the obstacles are too dense, the algorithm is not able to calculate effective results, and the paths calculated by RRT and APF are too close to the obstacles, which are not conducive to USV path planning.

These results of online path planning for USV show that: the proposed approach achieves the optimal path length in different environments, which demonstrates its practicality, generalizability, and robustness. Compared to the existing methods, the proposed approach can effectively find the optimal local path. The limitation is that the computing time is longer compared to the comparison algorithms, and it will grow exponentially with the size of the map.

5. Conclusions

In this paper, the DFQL for USV path planning algorithm is proposed, and simulation experiments are conducted according to the offline and online modes of USV in different maritime environments. The environment in the offline mode is completely known, and the static and dynamic obstacles in the online mode are partially known or completely unknown. Two modes are switched for practical work, which can effectively solve the USV path planning problem, saving energy and time. By combining APF with the Q-learning algorithm to initialize the Q-table and setting the static and dynamic reward function to provide the prior knowledge in the environment to the USV, the convergence speed and computation time of CQL are accelerated.

The obstacles in the actual working environment of the USV are not completely known. When the sensors detect static obstacles and dynamic obstacles around the USV and switch to online mode using the DFQL algorithm, simulation experiments and data analysis can show that the path and smoothness obtained by the DFQL algorithm are significantly better than other comparative algorithms, enabling the USV to implement obstacle avoidance.

In the future, we are going to study multi USV path planning. To reduce the calculation time and the coordination of multi USV, we will use deep reinforcement learning to study multi USV path planning.

CRediT authorship contribution statement

Bing Hao: and, Conceptualization, Methodology, Validation, Formal analysis, Writing – review & editing, Funding acquisition, All authors have read and agreed to the published version of the manuscript. **He Du:** Conceptualization, Software, Validation, Writing – original draft, Writing – review & editing, All authors have read and agreed to the published version of the manuscript. **Zheping Yan:** Validation, Supervision, Project administration, All authors have read and agreed to the published version of the manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

This work is partially supported by the Basic scientific research business cost scientific research project of Heilongjiang Provincial University (135509114).

References

Bakdi, A., Hentout, A., Boutami, H., Maoudj, A., Hachour, O., Bouzouia, B.J.R., Systems, A., 2017. Optimal path planning and execution for mobile robots using genetic algorithm and adaptive fuzzy-logic control. Robot. Autonom. Syst. 89, 95–109.

Caruana, R., Niculescu-Mizil, A., 2006. An empirical comparison of supervised learning algorithms. In: Proceedings of the 23rd International Conference on Machine Learning, pp. 161–168.

Cheng, C., Sha, Q., He, B., Li, G.J.O.E., 2021. Path planning and obstacle avoidance for AUV: a review. Ocean Eng. 235, 109355.

Chípuli, G.P., de la Mota, I.F.J.C.S.o.T.P., 2021. Analysis, design and reconstruction of a VRP model in a collapsed distribution network using simulation and optimization. Case Studies on Transport Policy 9 (4), 1440–1458.

Dai, X., Long, S., Zhang, Z., Gong, D.J.F.i.n., 2019. Mobile robot path planning based on ant colony algorithm with A* heuristic method. Front. Neurorob. 13, 15.

Eberhart, R., Kennedy, J., 1995. A new optimizer using particle swarm theory, MHS'95. In: Proceedings of the Sixth International Symposium on Micro Machine and Human Science. Ieee, pp. 39–43.

Goel, L.J.S.C., 2020. An extensive review of computational intelligence-based optimization algorithms. trends and applications 24 (21), 16519–16549.

Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018. Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: International Conference on Machine Learning. PMLR, pp. 1861–1870.

Han, J., Cho, Y., Kim, J., Kim, J., Son, N.s., Kim, S.Y.J.J.o.F.R., 2020. Autonomous collision detection and avoidance for ARAGON. USV: Development and field tests 37 (6), 987–1002.

Hao, B., Du, H., Zhao, J., Zhang, J., Wang, Q.J.C.I., Neuroscience, 2022. A Path-Planning Approach Based on Potential and Dynamic Q-Learning for Mobile Robots in Unknown Environment, 2022.

Hart, P.E., Nilsson, N.J., , Raphael, B.J.I.t.o.S.S., Cybernetics, 1968. A formal basis for the heuristic determination of minimum cost paths. IEEE Trans. Syst. Sci. Cybern. 4 (2), 100–107.

Jiang, L., Huang, H., Ding, Z.J.I.C.J.o.A.S., 2019. Path planning for intelligent robots based on deep. Q-learning with experience replay and heuristic knowledge 7 (4), 1179–1189.

Jin, J., Zhang, J., Shao, F., Lyu, Z., Wang, D.J.A.O.S., 2018. A novel ocean bathymetry technology based on an unmanned surface vehicle. Acta Oceanol. Sin. 37 (9), 99–106.

Kurowski, M., Thal, J., Damerius, R., Korte, H., Jeinsch, T.J.I.-P., 2019. Automated survey in very shallow water using an unmanned surface vehicle. IFAC-PapersOnLine 52 (21), 146–151.

LaValle, S.M., Kuffner Jr, J.J.J.T.i.j.o.r.r., 2001. Randomized kinodynamic planning. Int. J. Robot Res. 20 (5), 378–400.

Liu, X., Li, Y., Zhang, J., Zheng, J., Yang, C.J.I.A., 2019. Self-adaptive dynamic obstacle avoidance and path planning for USV under complex maritime environment. IEEE Access 7 7, 114945–114954.

Liu, Z., Zhang, Y., Yu, X., Yuan, C.J.A.R.i.C., 2016. Unmanned surface vehicles: an overview of developments and challenges. Annu. Rev. Control 41, 71–93.

Manley, J.E., 2008. Unmanned surface vehicles, 15 years of development, 2008. Ieee Oceans 1–4.

Mirjalili, S., Mirjalili, S.M., Lewis, A., 2014. Grey wolf optimizer. Adv. Eng. Software 69, 46–61.

Mirjalili, S.J.K.-b.s., 2015. Moth-flame optimization algorithm: a novel nature-inspired heuristic paradigm. Knowl. Base Syst. 89, 228–249.

Mirjalili, S.J.N.C., Applications, 2016. Dragonfly algorithm: a new meta-heuristic optimization technique for solving single-objective. discrete, and multi-objective problems 27 (4), 1053–1073.

Patle, B., Pandey, A., Parhi, D., Jagadeesh, A.J.D.T., 2019. A review: on path planning strategies for navigation of mobile robot. Defence Technology 15 (4), 582–606.

Powers, C., Hanlon, R., Schmale, D.G.J.R.S., 2018. Tracking of a fluorescent dye in a freshwater lake with an unmanned surface vehicle and an unmanned aircraft system. Rem. Sens. 10 (1), 81.

Sang, H., You, Y., Sun, X., Zhou, Y., Liu, F.J.O.E., 2021. The hybrid path planning algorithm based on improved A* and artificial potential field for unmanned surface vehicle formations. Ocean Eng. 223, 108709.

Schofield, R.T., Wilde, G.A., Murphy, R.R., 2018. Potential field implementation for move-to-victim behavior for a lifeguard assistant unmanned surface vehicle. In: 2018 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR). IEEE, pp. 1–2.

Shao, G., Ma, Y., Malekian, R., Yan, X., Li, Z.J.I.T.o.I.I., 2019. A novel cooperative platform design for coupled USV–UAV systems. IEEE Trans. Ind. Inf. 15 (9), 4913–4922.

Sung, I., Choi, B., Nielsen, P.J.I.J.o.I.M., 2021. On the training of a neural network for online path planning with offline path planning algorithms. Int. J. Inf. Manag. 57, 102142.

Sutresman, O., Syam, R., Asmal, S.J.I.J.T., 2017. Controlling unmanned surface vehicle rocket using GPS tracking method. Int. J. Technol 8, 709–718.

Tizhoosh, H.R., 2005. Opposition-based learning: a new scheme for machine intelligence, International conference on computational intelligence for modelling. In: Control and Automation and International Conference on Intelligent Agents, Web Technologies and Internet Commerce (CIMCA-IAWTIC'06). IEEE, pp. 695–701.

Tong, L., 2009. A speedup convergent method for multi-agent reinforcement learning. In: 2009 International Conference on Information Engineering and Computer Science. IEEE, pp. 1–4.

Watkins, C.J.C.H., 1989. Learning from Delayed Rewards.

Xue, K., Liu, C., Liu, H., Xu, R., Sun, Z., Lam, T.L., Qian, H., 2020. A two-stage automatic latching system for the USVs charging in disturbed berth. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, pp. 1748–1754.

Application Yan, R.-j., Pang, S., Sun, H.-b., Pang, Y.-j.J.J.o.M.S., 2010. Development and missions of unmanned surface vehicle. J. Mar. Sci. Appl. 9 (4), 451–457.

Yang, T., Guo, Y., Zhou, Y., Wei, S.J.I.A., 2019. Joint communication and control for small underactuated USV based on mobile computing technology. IEEE Access 7, 160610–160622.

Yao, Y.-l., Liang, X.-f., Li, M.-z., Yu, K., Chen, Z., Ni, C.-b., Teng, Y.J.C.O.E., 2021. Path planning method based on D* lite algorithm for unmanned surface vehicles in complex environments. China Ocean Eng. 35 (3), 372–383.

Yu, K., Liang, X.-f., Li, M.-z., Chen, Z., Yao, Y.-l., Li, X., Zhao, Z.-x., Teng, Y.J.O.E., 2021. USV path planning method with velocity variation and global optimisation based on AIS service platform. Ocean Eng. 236, 109560.